Bembank Defense Study Guide

Prepared for Espada | Review tonight and practice explaining in your own words.

This guide distinguishes the course template from the Bembank team-source copy in your Espada folder. Your teammates' presentation laptop may use a different configuration or version. Confirm it with them.

1. A 30-second introduction

"Good day. Our project is Bembank, a banking application developed for Integrative Programming Technologies 2. It integrates a user interface, a FastAPI backend, and a database. The frontend sends requests to the backend, which validates them, applies the rules, and accesses the database. We also packaged the interface as an Android APK for a mobile demonstration. This is a demo system, not a production bank."

Your role: explain your actual work honestly. For example: "I helped prepare and test the Android version and its connection to a backend using the laptop's network address." Add only contributions you really made.

2. The big picture

```
Android app / web browser
        |  HTTP requests and JSON
        v
FastAPI backend (run by Uvicorn)
        |  SQLAlchemy
        v
Database: users, accounts, transactions
        |
Response returns through backend to frontend
```

The frontend displays screens and collects input. The backend handles requests, authentication, authorization and business rules. The database stores records. The mobile app should not connect directly to the database.

In the template deployment, Nginx can sit between clients and Uvicorn, and Supervisor can manage the server process. Do not claim your group uses either one until you confirm the laptop setup.

3. Course template file structure

```
mytemplate-main/
  README.md               project overview and guide links
  labdemo2/               reference mini banking system
    app/                  reference FastAPI backend
    frontend/             reference web application
    frontendmobile/       reference customer mobile application
    demobackendmanual/    backend, database and deployment guides
    demofrontendmanual/   web frontend guides
  lessonmanual/           Git, JWT, Linux, Nginx and API lessons
  personal/               individual practice folders
```

Why is the template useful? It provides a reference architecture and lessons. Bembank is your group application, so explain what your code actually does instead of assuming every template detail applies.

4. Bembank study-copy file structure

The inspected copy is personal/espada/Bembank/team-source inside the Documents group repository. It is a separate study copy; the version on your teammate's laptop may differ.

```
team-source/
  app/                          FastAPI backend
    main.py                     creates app and registers routers
    core/                       settings, security, helper logic
    database/session.py         database engine and sessions
    models/                     SQLAlchemy database models
    schemas/                    request/response data validation
    routers/                    API endpoints and request logic
  frontend/                     Vue + Quasar application
    src/pages/                  login, dashboard and role pages
    src/components/             reusable UI components
    src/router/                 frontend routes and navigation
    src/stores/                 client-side state, including auth
    src/services/               API communication and server URL
    src-capacitor/android/      Android project used for APK build
  tests/                        backend checks
  scripts/                      support scripts, including demo seed
  requirements.txt              Python dependencies
  run-demo.sh                   starts the local study backend
  ESPADA-STUDY-README.md        setup notes for the study copy
```

What is app/main.py?

The backend entry point. It creates the FastAPI app, configures middleware and includes the routers. It also creates missing tables and initializes project defaults in the inspected copy.

What is the difference between models, schemas and routers?

Models describe database entities; schemas define and validate data sent to or returned by the API; routers define endpoints and their handling logic. A single feature often uses all three.

What is a database session?

A database session is the backend's working context for queries and changes. app/database/session.py configures the engine and supplies sessions to request handlers.

What do frontend pages, components and stores do?

Pages represent screens, components are reusable pieces of UI, and stores keep shared client-side state such as authentication information.

5. Know these specific files

app/routers/auth.py

Login, profile and session-related endpoints; in the inspected code login can lead to a six-digit PIN setup or verification.

app/core/security.py

Security helpers including authentication and role checks. Backend permissions matter even when the UI hides a page.

app/routers/customer_banking.py

Customer-facing account, transaction and transfer logic. It checks the current customer and filters accounts by ownership.

app/models/account.py and app/models/user.py

Database models for accounts and users. Foreign keys link records; inspect the actual model when asked about a specific field.

app/schemas/customer_banking.py

Defines data structures and validation used by customer banking endpoints.

frontend/src/pages/LoginPage.vue

Login screen, server-address field in the native app, and PIN-related UI.

frontend/src/services/api.js

Creates the Axios API client. In the native study build it can use the saved server address.

frontend/src/router/routes.js

Maps frontend URL paths to pages. Frontend routes are different from backend API endpoints.

frontend/src-capacitor/android/

Android project generated/used with Capacitor. The debug APK is an Android installation package.

scripts/seed_presentation.py

Provides fictional demonstration data for the separate study setup; don't assume it was run on the team laptop.

6. Follow a request from the phone

Login: LoginPage.vue collects credentials; the frontend API client sends a request to /auth/login; the auth router checks the credentials and can start a PIN challenge; the PIN endpoint verifies or sets up a six-digit PIN; after successful authentication, the app can send authenticated requests.

Account view: the app requests /customer/accounts/; the customer banking router identifies the current customer, queries accounts belonging to that customer through SQLAlchemy, and returns the response for the screen.

Transfer: the frontend sends transfer details to a customer transfer endpoint. The inspected backend checks the customer and source ownership, validates the destination and includes balance-related logic. Only demonstrate the exact flow you have tested on the team laptop.

7. Common panel questions

What problem does this project solve?

It demonstrates how customers and bank staff can interact with banking information through an integrated application, instead of treating the interface, backend and data as separate unrelated systems.

What is integrative programming?

Connecting separate components or technologies so they work together as one system. Here, the frontend, API and database communicate through defined interfaces.

What is an API and what is REST?

An API defines how software communicates. A REST-style API uses resources and HTTP methods, such as GET to retrieve data and POST to submit a request.

What are JSON and HTTP?

JSON is a common text format for structured data. HTTP is the protocol used to exchange requests and responses between the client and server.

What are GET, POST, PUT and DELETE?

They commonly retrieve, create, update and remove resources, respectively. The exact behavior depends on the endpoint.

What is SQLAlchemy?

A Python library that maps database tables to Python objects and provides tools for queries, sessions and transactions.

What is the difference between a primary key and a foreign key?

A primary key identifies a row in its table. A foreign key connects a row to a related record, such as an account associated with a customer.

What is a database transaction?

A group of database operations that can be committed together or rolled back on failure. It helps prevent a partially applied transfer.

What is authentication versus authorization?

Authentication checks who the user is. Authorization checks what that user is allowed to do.

What is a JWT?

A JSON Web Token is a signed token that can carry claims used by the server to identify a session. Signing protects integrity; it does not automatically encrypt the contents.

Why hash passwords and PINs?

Hashing avoids storing the original secret as plain text. At login, the server checks the provided value against its stored hash.

Why must the backend check roles and account ownership?

A user can bypass or modify a client interface. The server must enforce permission and ownership checks before returning data or performing an action.

What is CORS?

A browser security mechanism controlling whether a web page from one origin can access a different origin. It is not a replacement for authentication or authorization.

What does Capacitor do?

It packages the web frontend in an Android application and offers a bridge to supported native features.

What is an APK?

An Android installation package. Our prepared build is a debug APK for demonstration, not a production or Play Store release.

What is a reverse proxy?

A server such as Nginx receives client requests and forwards them to an application server, such as Uvicorn running FastAPI.

Why does 127.0.0.1 not connect the phone to the laptop?

It refers to the device making the request. On the phone it points back to the phone. Use the laptop's reachable network address, correct port and path.

Why might the phone still fail to connect?

The server may not be running, the address or port may be wrong, the firewall may block it, the devices may be on different networks, or the network may isolate clients.

How is a transfer protected from inconsistent updates?

Related changes should be committed in one database transaction, with appropriate validation and concurrency control. Do not claim production-grade concurrency or complete ledger guarantees without testing the actual deployment.

Is this connected to real GCash or a real bank?

Treat the features as simulated unless your team can prove a configured, authorized real provider integration and its actual transaction flow. A realistic-looking UI is not proof of integration.

Is this production ready?

No. It is a thesis demo. Real banking requires a security review, HTTPS, reliable transaction and reconciliation controls, operational monitoring, compliance, and verified provider integrations.

What is your contribution?

Explain what you personally did, how you tested it, and what remains unverified. Do not claim a teammate's code as your own.

How do Git branches and pull requests help?

They let members isolate changes, review them, and merge approved work into the shared branch. Describe your team's actual process, not just the ideal process.

What is the difference between MySQL and SQLite here?

The course template uses MySQL. The separate Espada study configuration defaults to a local SQLite file unless an environment variable overrides it. Ask the team which database their laptop uses.

What does an HTTP error mean?

400 generally signals an invalid request or business rule; 401 indicates missing or invalid authentication; 403 means forbidden; 404 means not found; 422 indicates validation failed; 500 indicates a server-side error. Exact behavior varies by endpoint.

8. Demo plan and troubleshooting

1. Ask the teammate to start the working backend and provide the laptop's address, port, and any URL path. 2. Connect both devices to the same Wi-Fi. 3. Enter that server address in the app. 4. Use an account that exists in that laptop's database. 5. Practice login, the PIN prompt, account view, and one verified transaction. 6. Keep

screenshots or a recording as backup.

Do not assume the old address or the Sofia demo account will exist on the team laptop. For the separate Espada study setup, the demo server uses port 5001 and fictional seeded accounts; its data is separate from the team laptop.

9. Rapid revision checklist

Can you explain the three layers? Point out main.py, a router, model, schema and frontend page? Trace login and an account request? Explain the laptop IP and why localhost fails? Explain authentication versus authorization? State clearly which features are simulated and which you tested? If yes, you are ready to practice aloud.

Honesty is a strength: if you do not know an implementation detail, say which component owns it and that you would check the code with your teammate. Do not invent an answer.

Sources and limits

Read locally: Downloads/mytemplate-main/README.md; labdemo2/demobackendmanual/core-architecture.md; labdemo2/frontend/BACKEND_FRONTEND_COMMUNICATION.md; labdemo2/frontendmobile/README.md and README.txt; Documents/sec11-g2/personal/espada/Bembank/team-source source files. Some template documents disagree about JWT expiry; verify the actual running code before stating a specific duration.

The group laptop, network, deployed configuration, live provider integrations, and complete phone-to-server demo were not verified for this guide. Local HTTP and demo credentials are not appropriate for real banking data.